Email Spam Detection using Machine Learning

MACHINE LEARNING CA-2 PROJECT REPORT

Submitted By:
Milind Pushp
Roll No. 202401100500105
Misty Jangid
Roll No. 202401100500106
Semester IV, Section B

Submitted To:
Dr. Shrankhla Saxena
Assistant Professor

Department of Computer Science and Information Technology
KIET Deemed to Be University

13 April, 2026

Abstract

Email communication is one of the most widely used forms of digital correspondence in both personal and professional settings. With this widespread adoption, spam emails have become a significant problem - cluttering inboxes, consuming bandwidth, and acting as a vector for phishing attacks and malware distribution.

This project implements and compares six classical machine-learning classifiers to automatically detect spam emails using the **Spambase dataset** from the UCI Machine Learning Repository. The dataset consists of 4,601 emails described by 57 features - 48 word-frequency attributes, 6 character-frequency attributes, and 3 capital-letter run-length statistics - along with a binary label indicating spam (1) or ham (0).

The methodology includes data loading, exploratory analysis, duplicate removal, missing-value imputation, feature scaling (StandardScaler), and an 80/20 stratified train-test split. Six models are trained: Logistic Regression, SVM (Linear kernel), SVM (RBF kernel), Random Forest, K-Nearest Neighbours, and Naïve Bayes. Performance is measured using accuracy, precision, recall, F1-score, and ROC-AUC.

The **Random Forest** classifier achieved the highest test accuracy of $\sim$**94.30%** with an ROC-AUC of $\sim$**0.99**, demonstrating its strong suitability for this task. The notebook is fully reproducible (due to set seed via `random_state=42`) and includes visualisations covering class distribution, feature importance, confusion matrices, ROC curves, and actual-vs-predicted comparisons.

Keywords: Spam Detection, Classification, Random Forest, SVM, Spambase, scikit-learn

Contents

1 Introduction **3**
- 1.1 Background . . . 3
- 1.2 Problem Statement . . . 3
- 1.3 Objectives . . . 3
- 1.4 Scope of the Project . . . 4

2 System Analysis **4**
- 2.1 Existing System . . . 4
- 2.2 Proposed System . . . 5
- 2.3 Feasibility Study . . . 5

3 Methodology **6**
- 3.1 Workflow Diagram . . . 6
- 3.2 Algorithms Used . . . 7
- 3.3 System Architecture . . . 8

4 Dataset Description **8**
- 4.1 Feature Groups . . . 9
- 4.2 Key Features (Top Spam Indicators) . . . 9

5 Implementation **9**
- 5.1 Tools & Technologies . . . 9
- 5.2 Steps of Implementation . . . 9
- 5.3 Code Snippets . . . 11

6 Results and Discussion **11**
- 6.1 Performance Summary . . . 11
- 6.2 Performance Metrics — Definitions . . . 12
- 6.3 Confusion Matrix — Random Forest (Best Model) . . . 12
- 6.4 Result Analysis . . . 12

7 Conclusion **13**
- 7.1 Summary of Work . . . 13
- 7.2 Key Achievements . . . 13
- 7.3 Future Improvements . . . 13

8 References **14**

1 Introduction

1.1 Background

Overview of the ML Concept

Spam detection is a classic **binary classification** problem in supervised machine learning. Given a set of labelled emails (spam / not-spam), the goal is to train a model that can generalise to new, unseen emails. The learning signal comes from statistical patterns in the email text - how often certain words or characters appear, and how the capital letters are used.

A variety of algorithms are applicable here, ranging from simple probabilistic methods (Naïve Bayes - the historical baseline for spam filters) to sophisticated ensemble methods (Random Forests, Gradient Boosting). Modern commercial spam filters often layer classical ML with neural networks and natural-language-processing techniques.

Domain Context

Email spam accounts for roughly **47.5% of all global email traffic** (Antispamengine.com, 2024). Beyond annoyance, spam is the primary delivery mechanism for:

- **Phishing** - credential theft through fake login pages.
- **Malware / ransomware** - via malicious attachments or links.
- **Advance-fee fraud** - financial scams.

Accurate, low-latency spam filters are therefore critical infrastructure for every email service provider.

1.2 Problem Statement

Given a numerical feature representation of an email derived from word-frequency statistics, character-frequency statistics, and capital-letter run-length statistics, **build a machine-learning model that can correctly classify whether an email is spam (1) or ham (0)** with high accuracy, precision, and recall.

The core challenge is to minimise **false negatives** (spam classified as ham, which is harmful) while keeping **false positives** (ham classified as spam, which annoys users) acceptably low.

1.3 Objectives

- Load and explore the UCI Spambase dataset (4,601 samples, 57 features).
- Perform exploratory data analysis (EDA) including class-distribution and feature-importance visualisations.
- Pre-process data: remove duplicates, impute missing values, standardise features.
- Train and evaluate six ML classifiers: Logistic Regression, SVM (Linear), SVM (RBF), Random Forest, KNN, and Naïve Bayes.

- Compare models across five metrics: Accuracy, Precision, Recall, F1-Score, ROC-AUC.
- Identify the best-performing model and generate detailed evaluation outputs (confusion matrix, ROC curve, feature importance).
- Document suggested improvements for future work.

1.4 Scope of the Project

Application Areas

- **Email providers** (Gmail, Outlook, Yahoo) - real-time inbox filtering.
- **Corporate IT security** - protecting employees from phishing.
- **Academic research** - baseline study for advanced NLP spam models.
- **Embedded security appliances** - lightweight rule-based + ML spam gates.

Limitations

- The Spambase dataset uses **bag-of-words** word-frequency features; it does not capture semantic meaning, header information, HTML structure, or image content.
- The word list is fixed and was curated in the late 1990s, certain modern spam phrases are absent.
- The model operates on pre-computed numerical features, it cannot directly classify raw email text without a feature-extraction pipeline.
- No temporal information is included, so concept drift (spam tactics evolve over time) is not handled.

2 System Analysis

2.1 Existing System

Traditional spam filters rely on:

Approach	Description	Limitation
Rule-based filters	Manually crafted keyword blacklists	Easily bypassed; require constant manual updates
Bayesian filters	Naïve Bayes on word counts	Good baseline, but struggles with adversarial spam (deliberate misspellings)
Blacklist / IP reputation	Block known spam servers	Only works for known senders; zero-day spam bypasses it
Content-based heuristics	Scoring based on phrases like "FREE!!!"	Rigid, easily evaded, high false-positive risk

Key limitations of existing systems:

- High maintenance cost (manual rule updates).
- Poor generalisation to novel spam campaigns.
- High false-positive rate (legitimate emails marked as spam).
- No learning from new patterns without human intervention.

2.2 Proposed System

Our proposed system trains multiple **supervised ML classifiers** end-to-end on a labelled dataset, allowing the model to **automatically learn discriminative patterns** without manual rule crafting.

Advantages:

- **Automatic feature learning** - the classifier discovers discriminative patterns from data.
- **Better generalisation** - ensemble methods like Random Forest are robust to overfitting.
- **Quantifiable performance** - metrics such as ROC-AUC and F1-score give objective quality measures.
- **Extensible** - new features (e.g., URL count, header fields) can be added without redesigning the system.
- **Reproducible** - the entire pipeline is scripted and version-controlled.

2.3 Feasibility Study

Technical Feasibility

The project uses the Python ecosystem (scikit-learn, pandas, NumPy, Matplotlib, Seaborn), all of which are open-source and run on any standard laptop or cloud environment. The dataset is small (< 1 MB), so no specialised hardware is required. Training all six models takes around 60 seconds on a mid-range CPU.

Economic Feasibility

All tools used are **free and open-source**. The dataset is publicly available from the UCI ML Repository. There are no licensing costs. Deployment on a cloud VM (if needed) would cost only a few cents per hour.

Operational Feasibility

The notebook-based implementation is easy for instructors and peers to run (Jupyter / VS Code). With minor adaptation, the trained model can be integrated into an email server via a REST API, making it operationally viable for real-world deployment.

3 Methodology

3.1 Workflow Diagram

```
Raw CSV File (spambase.data)
        |
        V
  Load Dataset
  (57 features + label)
        |
        V
  Exploratory Data Analysis
  (class distribution, word-frequency diff,
   missing-value check)
        |
        V
  Preprocessing
  +-----------------------------+
  | 1. Remove duplicates        |
  | 2. Impute missing values    |
  |    (column median)          |
  | 3. Train / Test split 80/20 |
  |    (stratified)             |
  | 4. StandardScaler           |
  +-----------------------------+
        |
        V
  Train 6 Classifiers
  +------------------------+
  | Logistic Regression    |
  | SVM (Linear kernel)    |
  | SVM (RBF kernel)       |
  | Random Forest          |
  | KNN (k = 5)            |
  | Naive Bayes            |
  +------------------------+
        |
        V
  Evaluate & Compare
  (Accuracy / Precision / Recall /
   F1 / ROC-AUC)
        |
        V
  Best Model -> Detailed Analysis
  (Confusion Matrix, ROC Curve,
   Feature Importance,
   Actual vs Predicted)
        |
```

```
        V
  Output & Conclusions
```

3.2 Algorithms Used

a) Logistic Regression

A linear model that estimates the probability of a sample belonging to class 1 (spam):

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \tag{1}$$

The decision boundary is a hyperplane in feature space. Trained with L2 regularisation (`max_iter=2000`).

b) Support Vector Machine (Linear & RBF)

Finds a maximum-margin hyperplane separating spam from ham.

- **Linear kernel**: decision surface is a hyperplane—fast, interpretable.
- **RBF kernel**: maps features to a higher-dimensional space, capturing non-linear patterns.

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\bigl(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\bigr) \tag{2}$$

c) Random Forest

An ensemble of `n_estimators=200` decision trees trained on bootstrap samples and random feature subsets. Final prediction is made by majority vote.

Why it works well here: the bag-of-words feature space contains many irrelevant features; random feature subsets at each split act as automatic regularisation.

d) K-Nearest Neighbours (k = 5)

Classifies each test email by a majority vote of its 5 nearest training neighbours (Euclidean distance in scaled feature space).

e) Gaussian Naïve Bayes

Assumes features are conditionally independent given the class and Gaussian-distributed:

$$P(x_i \mid y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right) \tag{3}$$

Historically the standard spam-filter baseline.

3.3 System Architecture

```
+---------------------------------------------+
|              USER / EMAIL SERVER            |
+-------------------+-------------------------+
                    | Raw email
                    V
+---------------------------------------------+
|        Feature Extraction Layer             |
|  * Word-frequency computation (48 feats)    |
|  * Char-frequency computation  (6 feats)    |
|  * Capital-run statistics      (3 feats)    |
+-------------------+-------------------------+
                    | 57-dim numeric vector
                    V
+---------------------------------------------+
|          Preprocessing Pipeline             |
|  * StandardScaler (fit on training data)    |
+-------------------+-------------------------+
                    | Scaled feature vector
                    V
+---------------------------------------------+
|         Trained ML Classifier               |
|  (Random Forest, 200 trees)                 |
+-------------------+-------------------------+
                    | Prediction: Spam / Ham
                    V
+---------------------------------------------+
|              OUTPUT / ACTION                |
|  * Route to inbox (Ham)                     |
|  * Move to Spam folder (Spam)               |
+---------------------------------------------+
```

4 Dataset Description

Source: UCI Machine Learning Repository—Spambase Data Set
(Creators: Mark Hopkins, Erik Reeber, George Forman, Jaap Suermondt—Hewlett-Packard Labs, 1999)

Property	**Value**
Total samples	4,601
Spam emails	1,813 (39.4%)
Ham emails	2,788 (60.6%)
Number of features	57
Missing values	None
File format	CSV (no header)
File size	∼702 KB

4.1 Feature Groups

#	Group	Features	Description
1	Word frequencies	48	Percentage of words in the email that match each word (e.g., `word_freq_free`, `word_freq_money`)
2	Character frequencies	6	Percentage of characters matching `;`, `(`, `[`, `!`, `$`, `#`
3	Capital run-length	3	Average, longest, and total length of uninterrupted sequences of capital letters
4	Label	1	`1` = Spam, `0` = Ham

4.2 Key Features (Top Spam Indicators)

Feature	Spam Mean	Ham Mean	Difference
`word_freq_free`	∼0.90%	∼0.07%	+0.83%
`word_freq_money`	∼0.54%	∼0.03%	+0.51%
`word_freq_remove`	∼0.40%	∼0.02%	+0.38%
`char_freq_exclaim`	∼0.54%	∼0.10%	+0.44%
`char_freq_dollar`	∼0.21%	∼0.01%	+0.19%
`capital_run_length_total`	∼567	∼110	+457

5 Implementation

5.1 Tools & Technologies

Tool / Library	Version	Purpose
Python	3.12	Core programming language
Jupyter Notebook / VS Code	—	Development environment
pandas	$\geq$ 2.x	Data loading & manipulation
NumPy	$\geq$ 1.26	Numerical computation
scikit-learn	$\geq$ 1.4	ML models, preprocessing, metrics
Matplotlib	$\geq$ 3.8	Plotting
Seaborn	$\geq$ 0.13	Statistical visualisations
Git / GitHub	—	Version control & collaboration

5.2 Steps of Implementation

Step 1 — Data Collection

The Spambase CSV file (`spambase.data`) was obtained from the UCI ML Repository. Because it has no header row, column names were manually assigned based on the UCI documentation (48 word-frequency names, 6 character-frequency names, 3 capital run-length names, and `label`).

Step 2 — Preprocessing

```python
# Remove duplicates
df = df.drop_duplicates()

# Impute missing values (in case any exist after future updates)
df = df.fillna(df.median(numeric_only=True))

# Feature / target split
X = df.drop("label", axis=1)
y = df["label"]

# Stratified 80/20 train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Feature scaling
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)
```

Why StandardScaler? Distance-based models (KNN, SVM) are very sensitive to feature magnitude. The capital run-length features have values in the hundreds, while word-frequency features are in the range 0–100. Scaling ensures no single feature dominates due to its scale.

Step 3 — Model Training

```python
MODELS = {
    "Logistic Regression" : LogisticRegression(max_iter=2000, random_state=42),
    "SVM (Linear)"        : SVC(kernel="linear", probability=True, random_state=42),
    "SVM (RBF)"           : SVC(kernel="rbf",    probability=True, random_state=42),
    "Random Forest"       : RandomForestClassifier(n_estimators=200, random_state=42),
    "KNN (k=5)"           : KNeighborsClassifier(n_neighbors=5),
    "Naive Bayes"         : GaussianNB(),
}

for name, model in MODELS.items():
    model.fit(X_train_sc, y_train)
```

Step 4 — Testing & Evaluation

```python
for name, model in MODELS.items():
    y_pred = model.predict(X_test_sc)
    y_prob = model.predict_proba(X_test_sc)[:, 1]
    print(f"{name}: Accuracy={accuracy_score(y_test, y_pred)*100:.2f}%"
          f"ROC-AUC={roc_auc_score(y_test, y_prob):.4f}")
```

5.3 Code Snippets

Confusion Matrix Heatmap

```
cm = confusion_matrix(y_test, y_pred_best)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Ham", "Spam"],
            yticklabels=["Ham", "Spam"])
plt.title(f"Confusion Matrix - {best_name}")
plt.ylabel("Actual");  plt.xlabel("Predicted")
plt.show()
```

ROC Curve (All Models)

```
for name, model in MODELS.items():
    y_prob = model.predict_proba(X_test_sc)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    auc = roc_auc_score(y_test, y_prob)
    plt.plot(fpr, tpr, label=f"{name} (AUC={auc:.3f})")
plt.plot([0,1],[0,1],"k--")
plt.xlabel("FPR");  plt.ylabel("TPR")
plt.legend(fontsize=8)
plt.show()
```

Feature Importance (Random Forest)

```
feat_imp = pd.Series(rf_model.feature_importances_, index=X.columns)
feat_imp.sort_values(ascending=False).head(20).sort_values().plot(kind=
    "barh")
plt.title("Top 20 Feature Importances - Random Forest")
plt.show()
```

6 Results and Discussion

6.1 Performance Summary

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Random Forest	**94.30%**	**94.48%**	**94.30%**	**94.31%**	**∼0.990**
SVM (Linear)	93.59%	93.65%	93.59%	93.57%	∼0.985
Logistic Regression	93.47%	93.54%	93.47%	93.45%	∼0.983
SVM (RBF)	93.23%	93.31%	93.23%	93.21%	∼0.982
KNN (k=5)	91.33%	91.55%	91.33%	91.33%	∼0.965
Naïve Bayes	82.90%	84.20%	82.90%	82.47%	∼0.940

(Metric values are approximate; exact values may vary slightly due to dataset preprocessing.)

Metric	Formula	Meaning
Accuracy	(TP + TN) / Total	Overall correct predictions
Precision	TP / (TP + FP)	Of all predicted spam, how many are actually spam?
Recall	TP / (TP + FN)	Of all actual spam, how many were caught?
F1-Score	$2 \times (P \times R)/(P + R)$	Harmonic mean of Precision and Recall
ROC-AUC	Area under ROC curve	Overall ranking quality across all thresholds

6.2 Performance Metrics — Definitions

Note on false negatives (FN): In spam detection, failing to detect spam (FN) is generally worse than falsely flagging a legitimate email (FP). A higher **Recall** on the Spam class is therefore the priority metric in practice.

6.3 Confusion Matrix — Random Forest (Best Model)

```
                Predicted
                Ham    Spam
Actual  Ham   [ 485     21 ]    -> 95.9% of ham correctly identified
        Spam  [  27    309 ]    -> 91.9% of spam correctly identified
```

- **False Positives (21):** 21 legitimate emails incorrectly flagged as spam.
- **False Negatives (27):** 27 spam emails that slipped through to the inbox.

6.4 Result Analysis

1. **Random Forest** is the clear winner, combining high accuracy with the highest ROC-AUC (∼0.99), showing near-perfect ranking ability across all decision thresholds.
2. **Linear models** (Logistic Regression, SVM-Linear) perform surprisingly close to Random Forest (∼93–94%) because the Spambase feature space is largely linearly separable once scaled.
3. **Naïve Bayes** (82.9%) underperforms because it assumes feature independence—in practice, features like `word_freq_free` and `word_freq_money` are highly correlated in spam, violating this assumption.
4. **KNN** (91.3%) suffers from the curse of dimensionality; with 57 features, distance metrics become less meaningful without further dimensionality reduction.
5. **Top spam-discriminating features** (from Random Forest feature importance): `capital_run_length_total`, `word_freq_free`, `char_freq_exclaim`, `char_freq_dollar`, `word_freq_remove`, `word_freq_your`.

7 Conclusion

This project successfully demonstrated the application of multiple supervised machine-learning classifiers to the email spam detection problem using the UCI Spambase dataset.

7.1 Summary of Work

- Loaded and explored 4,601 email samples described by 57 engineered features.
- Performed preprocessing including duplicate removal, median imputation, stratified splitting, and StandardScaler normalisation.
- Trained and compared **six classifiers**: Logistic Regression, SVM (Linear), SVM (RBF), Random Forest, KNN, and Naïve Bayes.
- Evaluated all models using five metrics: Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
- Generated comprehensive visualisations: class distribution, word-frequency analysis, accuracy comparison, confusion matrix, ROC curves, and feature importance.

7.2 Key Achievements

- Achieved a best test accuracy of **94.30%** and ROC-AUC of $\sim$**0.99** with Random Forest.
- Identified `capital_run_length_total`, `word_freq_free`, and `char_freq_exclaim` as the strongest spam indicators.
- Produced a clean, reproducible Jupyter Notebook (`Email_Spam_Detection_Improved.ipynb`) that follows software engineering best practices (single `RANDOM_STATE`, modular code, inline explanations).

7.3 Future Improvements

Improvement	**Expected Benefit**
Hyperparameter tuning (GridSearchCV)	+1–3% accuracy
10-fold cross-validation	More reliable performance estimate
SMOTE / class-weight balancing	Improved recall on spam class
log1p transform on skewed features	Better KNN / SVM performance
Ensemble stacking	Marginal accuracy gain
Feature selection (SelectKBest, PCA)	Faster inference, interpretability
Cost-sensitive learning	Minimise false-negative rate
Model serialisation (joblib)	Production deployment readiness

8 References

1. M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, "Spambase Data Set," UCI Machine Learning Repository, 1999. [Online]. Available: https://archive.ics.uci.edu/ml/datasets/Spambase

2. T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York, USA: Springer, 2009.

3. F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

4. L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

5. C. Cortes and V. Vapnik, "Support-Vector Networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

6. I. Androutsopoulos, J. Koutsias, K. V. Chandrinos, and C. D. Spyropoulos, "An Experimental Comparison of Naive Bayesian and Keyword-Based Anti-Spam Filtering with Encrypted Personal E-Mail Messages," *Proc. 23rd ACM SIGIR*, pp. 160–167, 2000.

7. Antispamengine.com, "Spam Statistics," 2024. [Online]. Available: https://antispamengine.com/spam-statistics/

Report prepared for Machine Learning CA-2 Assessment, KIET Deemed to Be University, 13 April 2026.